

NYC TLC Trip Records Data Pipeline

Industry-Grade Project Plan

PySpark | Snowflake | dbt | Apache Airflow

Dataset: April 2026

Table of Contents

1. Architecture Overview
2. Dataset Details & Explanations
 - 2.1 Yellow Taxi Trips
 - 2.2 Green Taxi Trips
 - 2.3 FHV Trips
 - 2.4 HVFHV Trips
 - 2.5 Taxi Zone Lookup (Seed)
 - 2.6 Manual Seed Tables
 - 2.7 Dataset Summary Table
3. Project Phases
4. Requirements
 - 4.1 R1 - PySpark Ingestion
 - 4.2 R2 - Snowflake Setup
 - 4.3 R3 - dbt Transformation
 - 4.4 R4 - Airflow Orchestration
 - 4.5 R5 - Data Quality
 - 4.6 R6 - Non-Functional
5. dbt Model Map
6. Airflow DAG Flow
7. Folder Structure
8. File-by-File Explanation
 - 8.1 Airflow Files
 - 8.2 Spark Files
 - 8.3 dbt Files
 - 8.4 Snowflake & Root Files
9. Key Concepts Covered

1. Architecture Overview

This project builds an industry-grade data pipeline ingesting NYC TLC April 2026 trip records through four layers: ingestion (PySpark), landing (Snowflake RAW), transformation (dbt), and orchestration (Apache Airflow).

```
[NYC TLC CloudFront CDN]
  Parquet files (Yellow, Green, FHV, HVFHV)
    |
    v PySpark Ingestion Layer
  - Download April 2026 Parquet files
  - Enforce schema, validate, deduplicate
  - Derive trip_duration_minutes, avg_speed_mph
  - Write partitioned Parquet --> S3 landing zone
    |
    v Snowflake -- RAW schema (COPY INTO)
raw_yellow_trips | raw_green_trips
raw_fhv_trips    | raw_hvfhv_trips
    |
    v dbt Transformation
STAGING --> INTERMEDIATE --> DIM + FCT --> MARTS
    |
    v Apache Airflow Orchestration
daily_tlc_pipeline DAG (06:00 UTC)
incremental_events DAG (hourly)
dbt_snapshots DAG (nightly)
    |
    v BI / Reporting
dbt docs + exposures (Metabase / Tableau ready)
```

2. Dataset Details & Explanations

All files are sourced from the NYC TLC Trip Record Data page. April 2026 files follow the naming convention: <type>_tripdata_2026-04.parquet

Download URLs

Trip Type	Download URL
Yellow Taxi	https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2026-04.parquet
Green Taxi	https://d37ci6vzurychx.cloudfront.net/trip-data/green_tripdata_2026-04.parquet
FHV	https://d37ci6vzurychx.cloudfront.net/trip-data/fhv_tripdata_2026-04.parquet
HVHV	https://d37ci6vzurychx.cloudfront.net/trip-data/hvhv_tripdata_2026-04.parquet
Zone Lookup	https://d37ci6vzurychx.cloudfront.net/misc/taxi_zone_lookup.csv

2.1 Yellow Taxi Trips - yellow_tripdata_2026-04.parquet

The most iconic NYC dataset. Yellow taxis are the traditional metered cabs that can be hailed from the street anywhere in New York City. They are the only vehicles legally allowed to pick up street-hail passengers in Manhattan and at the airports.

Collected by: Creative Mobile Technologies (VendorID=1) and VeriFone (VendorID=2) via in-cab metering hardware.

Estimated volume: ~3-4 million rows for April 2026. One row = one completed trip.

What makes it unique

- Richest schema of all four datasets - full fare breakdown (fare, tip, tolls, surcharges, airport fees)
- Only dataset where tip_amount is reliably populated (credit card tips only; cash tips not captured)
- Has RatecodeID: 1=Standard, 2=JFK flat fare, 3=Newark, 4=Nassau, 5=Negotiated, 6=Group ride
- Has store_and_fwd_flag: Y if trip was stored in cab memory before transmission (no cellular signal)
- congestion_surcharge (\$2.50) applies to trips through Manhattan south of 96th Street
- airport_fee (\$1.25) applies to JFK and LaGuardia pickups

Yellow Taxi Schema

Column	Type	Description
VendorID	INT	1=CMT, 2=VeriFone
tppe_pickup_datetime	TIMESTAMP	Pickup time (tppe = Taxicab Passenger Enhancement Program)
tppe_dropoff_datetime	TIMESTAMP	Dropoff time
passenger_count	INT	Number of passengers (driver-entered)
trip_distance	FLOAT	Trip distance in miles (metered)
RatecodeID	INT	1=Standard, 2=JFK, 3=Newark, 4=Nassau, 5=Negotiated, 6=Group
store_and_fwd_flag	STRING	Y/N - was trip stored in cab memory before sending
PULocationID	INT	Pickup taxi zone ID (FK to taxi_zone_lookup)
DOLocationID	INT	Dropoff taxi zone ID (FK to taxi_zone_lookup)
payment_type	INT	1=Credit, 2=Cash, 3=No charge, 4=Dispute, 5=Unknown, 6=Voided
fare_amount	FLOAT	Metered fare amount
extra	FLOAT	Rush hour and overnight surcharges
mta_tax	FLOAT	MTA tax (\$0.50)
tip_amount	FLOAT	Tip amount (credit card only)

NYC TLC Data Pipeline - Project Plan

tolls_amount	FLOAT	Total tolls paid
improvement_surcharge	FLOAT	\$0.30 improvement surcharge
total_amount	FLOAT	Total amount charged to passenger
congestion_surcharge	FLOAT	NYC congestion pricing surcharge
airport_fee	FLOAT	JFK/LaGuardia airport pickup fee

Common Data Quality Issues

- Rows with pickup_datetime outside April 2026 (vendor transmission delays)
- passenger_count = 0 (driver forgot to enter)
- Negative fare_amount or total_amount (fare corrections/reversals)
- tip_amount > 0 when payment_type = 2 (cash - impossible, data entry error)

2.2 Green Taxi Trips - green_tripdata_2026-04.parquet

Green taxis (Boro Taxis) were introduced in 2013 to serve the outer boroughs (Bronx, Brooklyn, Queens, Staten Island) and Upper Manhattan (above 96th St East, above 110th St West). They cannot pick up street-hail passengers in lower Manhattan or at airports.

Estimated volume: ~500,000-700,000 rows. Significantly fewer than Yellow due to geographic restrictions.

What makes it unique

- Schema nearly identical to Yellow but datetime columns are lpep_pickup_datetime / lpep_dropoff_datetime (lpep = Livery Passenger Enhancement Program)
- Has trip_type column (1=Street-hail, 2=Dispatch) which Yellow lacks
- No airport_fee column - Green taxis cannot pick up at JFK or LaGuardia
- Generally shorter trips and lower fares than Yellow (outer borough geography)
- lpep_ columns must be renamed to pickup_datetime/dropoff_datetime in staging to match Yellow naming convention for the union in the intermediate layer

Common Data Quality Issues

- Same issues as Yellow (passenger_count=0, fare corrections)
- Some rows have PULocationID in Manhattan below 96th St - technically illegal pickups, worth flagging

2.3 FHV Trips - fhv_tripdata_2026-04.parquet

FHV covers traditional for-hire vehicles: black cars, livery cars, and luxury sedans pre-booked through a dispatch base. This category excludes high-volume app-based services (Uber/Lyft) which are in the HVFHV dataset.

Estimated volume: ~1-2 million rows.

What makes it unique

- NO fare data - FHV trips are not metered. Pricing is negotiated. The TLC does not collect fare information for this category.
- Has dispatching_base_num: the TLC license number of the dispatching base (e.g. B02512)
- Has SR_Flag: 1 = part of a shared/pooled ride
- Much simpler schema than Yellow/Green because of missing fare columns
- Useful for analysing trip volume and geographic coverage of black car services

Common Data Quality Issues

- Many NULL values in PULocationID / DOLocationID (location data not always reported)
- pickup_datetime and dropoff_datetime sometimes identical (zero-duration trips - data entry errors)

2.4 HVFHV Trips - fhvhv_tripdata_2026-04.parquet

HVFHV covers app-based ride-hailing companies completing 10,000+ trips per day in NYC: Uber (HV0003), Lyft (HV0005), Via (HV0004), and Juno/Gett (HV0002). Separated from FHV in 2019 when the TLC created a dedicated licensing category for app-based platforms.

Estimated volume: ~15-20 million rows. The largest file by a significant margin - Uber and Lyft alone handle millions of trips per month.

What makes it unique

- The ONLY dataset that includes driver_pay - allows analysis of platform economics and driver earnings
- Has request_datetime and on_scene_datetime in addition to pickup/dropoff - can calculate wait times
- base_passenger_fare vs driver_pay gap reveals platform commission rates

- `shared_request_flag` vs `shared_match_flag`: distinguishes pool requests from actual pool matches
- `hvfhs_license_num` identifies the platform (HV0003=Uber, HV0005=Lyft)
- `wav_request_flag` / `wav_match_flag`: wheelchair-accessible vehicle requests and matches

Common Data Quality Issues

- Very large file - April 2026 could be 1-2 GB; needs careful Spark partitioning
- `on_scene_datetime` sometimes before `pickup_datetime` (data entry errors)
- `driver_pay` occasionally negative (corrections/adjustments)

2.5 Taxi Zone Lookup - taxi_zone_lookup.csv (Seed)

A 265-row static reference table mapping the integer LocationID (used as PULocationID and DOLocationID in all four trip datasets) to human-readable zone names, boroughs, and service zone categories.

Source: https://d37ci6vzurychx.cloudfront.net/misc/taxi_zone_lookup.csv

Column	Description
LocationID	Integer 1-265, the zone identifier used in all trip datasets
Borough	NYC borough: Manhattan, Brooklyn, Queens, Bronx, Staten Island, EWR
Zone	Human-readable zone name (e.g. JFK Airport, Times Sq/Theatre District)
service_zone	Category: Yellow Zone, Boro Zone, Airports, EWR

Special zones to know

- Zone 132 = JFK Airport
- Zone 138 = LaGuardia Airport
- Zone 1 = Newark Airport (EWR) - technically in New Jersey
- Zone 264 = Unknown, Zone 265 = NV (Not a valid NYC zone)

2.6 Manual Seed Tables (vendors, rate_codes, payment_types)

Three small CSV files created manually to decode integer codes in the Yellow and Green trip datasets.

File	Rows	Purpose
vendors.csv	2	Maps VendorID 1/2 to Creative Mobile Technologies / VeriFone Inc.
rate_codes.csv	6	Maps RatecodeID 1-6 to Standard, JFK, Newark, Nassau, Negotiated, Group
payment_types.csv	6	Maps payment_type 1-6 to Credit, Cash, No charge, Dispute, Unknown, Voided

Note: JFK flat-rate trips (RatecodeID=2) are priced at a fixed \$52 + tolls regardless of distance. Filter these out when calculating fare-per-mile averages to prevent statistical distortion.

2.7 Dataset Summary Table

Dataset	Format	Est. Rows (Apr 2026)	Fare Data	Location Data
Yellow Taxi	Parquet	3-4 million	Yes (full)	Yes
Green Taxi	Parquet	500K-700K	Yes (full)	Yes
FHV	Parquet	1-2 million	No	Partial
HVFHV	Parquet	15-20 million	Partial (driver pay)	Yes
Taxi Zone Lookup	CSV	265 rows	N/A	N/A
vendors.csv	CSV (manual)	2 rows	N/A	N/A
rate_codes.csv	CSV (manual)	6 rows	N/A	N/A
payment_types.csv	CSV (manual)	6 rows	N/A	N/A

3. Project Phases

Phase 1: Infrastructure Setup

Set up Snowflake (warehouse, databases, roles, schemas), S3 bucket as landing zone, Airflow via Docker Compose, dbt project scaffold, and PySpark local environment.

Phase 2: PySpark Ingestion Layer

Build four PySpark jobs (one per trip type) to download files, enforce schema, validate rows, derive new columns, and write partitioned Parquet to S3.

Phase 3: Snowflake Landing

Run COPY INTO commands to load S3 Parquet files into RAW schema tables. Create file format objects, external stages, and named file formats.

Phase 4: dbt Transformation Layer

Build staging views, intermediate union model, dimension tables from seeds, incremental fct_trips, and three mart aggregation tables.

Phase 5: Airflow Orchestration

Build three DAGs: daily_tlc_pipeline (06:00 UTC), incremental_events (hourly), and dbt_snapshots (nightly). Wire all tasks with retries, sensors, and Slack alerts.

Phase 6: Data Quality & Observability

Add dbt generic tests, dbt_expectations statistical tests, row count logging in Spark, and Slack failure callbacks in Airflow.

Phase 7: Documentation & Lineage

Write column descriptions in all .yaml files, declare BI exposures, run dbt docs generate, and write component READMEs.

4. Requirements

4.1 R1 - PySpark Ingestion Requirements

ID	Requirement
R1.1	Download all 4 trip type Parquet files for April 2026 from TLC CloudFront URLs
R1.2	Enforce explicit schema on read - no schema inference
R1.3	Reject rows where pickup_datetime > dropoff_datetime (invalid trips)
R1.4	Reject rows where total_amount <= 0 or trip_distance < 0
R1.5	Deduplicate rows within each file on a composite key
R1.6	Derive trip_duration_minutes = dropoff - pickup
R1.7	Derive avg_speed_mph = trip_distance / (trip_duration_minutes / 60) - null if duration = 0
R1.8	Add source_file column (filename) and ingested_at timestamp
R1.9	Write output as Parquet, partitioned by trip_type and pickup_date to S3
R1.10	Log row counts: raw -> after dedup -> after validation -> written

4.2 R2 - Snowflake Setup Requirements

ID	Requirement
R2.1	Create separate schemas: RAW, STAGING, MARTS
R2.2	Create external stage pointing to S3 landing zone
R2.3	Create named file format object for Parquet
R2.4	COPY INTO each raw table using MATCH_BY_COLUMN_NAME
R2.5	Enable COPY INTO with PURGE = FALSE (keep source files)
R2.6	Create a dedicated TRANSFORM role with least-privilege grants
R2.7	Use auto-suspend warehouse (5-min idle) for cost control

4.3 R3 - dbt Transformation Requirements

ID	Requirement
R3.1	Staging models must be views; dim/fct/mart models must be tables
R3.2	fct_trips must be an incremental model keyed on trip_id (surrogate key)
R3.3	dim_taxi_zones loaded from taxi_zone_lookup.csv seed
R3.4	int_all_trips_unioned must union Yellow + Green + HVFHV with a trip_type discriminator column
R3.5	Every model must have column-level descriptions in .yaml files
R3.6	All primary keys must have unique + not_null tests
R3.7	All foreign keys (e.g. PULocationID -> dim_taxi_zones) must have relationships tests
R3.8	fare_amount, total_amount must have dbt_expectations.expect_column_values_to_be_between tests
R3.9	dim_vendors, dim_rate_codes, dim_payment_types must be static seed tables
R3.10	mart_daily_summary must aggregate: trip count, avg fare, avg distance, avg duration, total revenue per day
R3.11	mart_zone_performance must show top/bottom pickup zones by trip volume and revenue
R3.12	mart_revenue_breakdown must split revenue by: fare, tip, tolls, surcharges, airport fees
R3.13	Build a custom macro safe_divide(numerator, denominator) for null-safe division
R3.14	Use dbt_utils.generate_surrogate_key for trip_id on fct_trips
R3.15	Add dbt docs exposure block for the final marts (BI layer)

4.4 R4 - Airflow Orchestration Requirements

ID	Requirement
R4.1	daily_tlc_pipeline DAG must run at 06:00 UTC daily
R4.2	DAG must use SparkSubmitOperator for all PySpark jobs
R4.3	DAG must use SnowflakeOperator for COPY INTO tasks
R4.4	DAG must use BashOperator for all dbt commands
R4.5	All tasks must have retries=3 and retry_delay=5min
R4.6	Use S3KeySensor to confirm files exist in S3 before COPY INTO
R4.7	dbt test must run after dbt run; pipeline must fail if tests fail
R4.8	All credentials (Snowflake, S3, Slack) stored in Airflow Connections - never in code
R4.9	on_failure_callback must send a Slack alert on any task failure
R4.10	DAG must be tagged with owner, dataset, env
R4.11	SLA miss alert if full pipeline exceeds 90 minutes

4.5 R5 - Data Quality Requirements

ID	Requirement
R5.1	Row count check: raw row count must match Snowflake landing count (logged in Airflow)
R5.2	No trip should have pickup_datetime outside April 2026 in the staging layer
R5.3	passenger_count must be between 0 and 9
R5.4	trip_distance must be between 0 and 500 miles
R5.5	total_amount must be between 0 and 5000
R5.6	PULocationID and DOLocationID must exist in dim_taxi_zones
R5.7	Null rate: alert if any key column exceeds 5% nulls

4.6 R6 - Non-Functional Requirements

ID	Requirement
R6.1	All code in Git - monorepo with /airflow, /spark, /dbt folders
R6.2	Secrets managed via .env (local) - never committed to Git
R6.3	Docker Compose for local Airflow (webserver + scheduler + postgres metadata DB)
R6.4	README.md per component with setup and run instructions
R6.5	dbt docs generate must produce full lineage graph
R6.6	Snowflake warehouse must auto-suspend after 5 min idle

5. dbt Model Map

```

models/
|- staging/
|   |- source.yml
|   |- stg_yellow_trips.sql      (view)
|   |- stg_green_trips.sql      (view)
|   |- stg_fhv_trips.sql        (view)
|   `-- stg_hvfhv_trips.sql     (view)
|- intermediate/
|   `-- int_all_trips_unioned.sql (view - union of yellow+green+hvfhv)
|- dim/
|   |- dim_taxi_zones.sql        (table - from seed)
|   |- dim_vendors.sql          (table - from seed)
|   |- dim_rate_codes.sql        (table - from seed)
|   `-- dim_payment_types.sql    (table - from seed)
|- fct/
|   `-- fct_trips.sql            (incremental table, surrogate key)
|- marts/
|   |- mart_daily_summary.sql    (table)
|   |- mart_zone_performance.sql (table)
|   `-- mart_revenue_breakdown.sql (table)
|- macros/
|   `-- safe_divide.sql
`-- seeds/
    |- taxi_zone_lookup.csv
    |- vendors.csv
    |- rate_codes.csv
    `-- payment_types.csv

```

6. Airflow DAG Flow

daily_tlc_pipeline (06:00 UTC)

```

spark_ingest_yellow  --|
spark_ingest_green   --|-- (parallel SparkSubmit)
spark_ingest_fhv     --|
spark_ingest_hvfhv   --|
    |
s3_sensor (S3KeySensor - wait for files)
    |
snowflake_copy_yellow --|
snowflake_copy_green  --|-- (parallel COPY INTO)
snowflake_copy_fhv    --|
    |
dbt_seed
dbt_run_staging
dbt_run_intermediate
dbt_run_dim
dbt_run_fct
dbt_test              <-- fails pipeline if tests fail
dbt_run_marts
notify_success        (SlackWebhookOperator)

```

7. Folder Structure

```
nyc_tlc_pipeline/  
|- airflow/  
|   |- dags/  
|   |   `-- daily_tlc_pipeline.py  
|   |- plugins/  
|   |- docker-compose.yml  
|   `-- requirements.txt  
|- spark/  
|   |- jobs/  
|   |   |- ingest_yellow.py  
|   |   |- ingest_green.py  
|   |   |- ingest_fhv.py  
|   |   `-- ingest_hvfhv.py  
|   |- utils/  
|   |   |- spark_session.py  
|   |   |- schema_registry.py  
|   |   `-- data_quality.py  
|   `-- requirements.txt  
|- dbt/  
|   `-- nyc_tlc/  
|       |- models/  
|       |- macros/  
|       |- seeds/  
|       |- snapshots/  
|       |- tests/  
|       `-- dbt_project.yml  
|- snowflake/  
|   `-- setup.sql  
|- .env.example  
`-- README.md
```

8. File-by-File Explanation

8.1 Airflow Files

airflow/dags/daily_tlc_pipeline.py

The main Airflow DAG file. Defines the entire pipeline as a directed acyclic graph - task definitions, dependencies, schedule (0 6 * * *), retries, SLA, Slack callbacks, and tags. This is the entry point Airflow reads to know what to run and in what order.

airflow/plugins/

Directory for custom Airflow plugins - e.g. custom operators, hooks, or sensors specific to this project (e.g. a TLCDownloadOperator). Empty initially; add plugins here rather than polluting the DAG file.

airflow/docker-compose.yml

Defines the local Airflow environment using Docker: airflow-webserver, airflow-scheduler, postgres (metadata DB), and optionally redis (CeleryExecutor). Mounts dags/, plugins/, and logs/ as volumes.

airflow/requirements.txt

Python dependencies for the Airflow environment: apache-airflow[snowflake,amazon,slack] and apache-airflow-providers-apache-spark.

8.2 Spark Files

spark/jobs/ingest_yellow.py

PySpark job for Yellow Taxi data. Downloads yellow_tripdata_2026-04.parquet, applies the explicit Yellow schema, runs validation rules (R1.3-R1.5), derives trip_duration_minutes and avg_speed_mph, appends source_file and ingested_at metadata, and writes partitioned Parquet to S3.

spark/jobs/ingest_green.py

Same structure as ingest_yellow.py but for Green Taxi data. Handles the lpep_datetime column naming difference and additional trip_type column. Schema handled separately in schema_registry.py.

spark/jobs/ingest_fhv.py

PySpark job for FHV data. Simpler schema - no fare/payment columns. Applies validation and writes to S3. Revenue columns are set to NULL to allow optional union with other trip types.

spark/jobs/ingest_hvfhv.py

PySpark job for HVFHV (Uber/Lyft) data. The largest file (~1-2 GB). Handles HVFHV-specific columns: hvfhs_license_num, originating_base_num, request_datetime, on_scene_datetime, driver_pay, base_passenger_fare, tips. Full validation, enrichment, and partitioned write to S3.

spark/utls/spark_session.py

Factory function that creates and returns a configured SparkSession. Centralises Spark config: app name, S3 credentials (from env vars), shuffle partitions, serializer. All job files import get_spark_session() from here.

spark/utls/schema_registry.py

Defines explicit PySpark StructType schemas for all four trip types as module-level constants (YELLOW_SCHEMA, GREEN_SCHEMA, FHV_SCHEMA, HVFHV_SCHEMA). Prevents schema inference in production.

spark/utils/data_quality.py

Helper functions used across all job files: `log_row_counts()`, `validate_date_range()`, `assert_no_duplicates()`, `check_null_rate()`. Centralises all data quality logic.

8.3 dbt Files

dbt_project.yml

The dbt project configuration file. Defines project name (nyc_tlc), profile name, model paths, seed paths, snapshot paths, and default materializations per folder (staging=view, dim/fct/marts=table).

models/staging/source.yml

Declares the raw Snowflake tables as dbt sources. Registers RAW.raw_yellow_trips, raw_green_trips, raw_fhv_trips, raw_hvfhv_trips. Enables source() references and source freshness checks.

models/staging/stg_yellow_trips.sql

Thin view over raw_yellow_trips. Renames columns to snake_case, casts datetime columns, adds trip_type='yellow' literal, and filters rows where pickup_datetime > dropoff_datetime.

models/staging/stg_green_trips.sql

Same as stg_yellow_trips.sql but renames lpep_datetime columns to the unified pickup_datetime/dropoff_datetime naming convention so all staging models share the same column names for the intermediate union.

models/staging/stg_fhv_trips.sql

Thin view over raw_fhv_trips. Renames and casts columns. Revenue columns filled with NULL since FHV has no fare data.

models/staging/stg_hvfhv_trips.sql

Thin view over raw_hvfhv_trips. Renames Uber/Lyft-specific columns (base_passenger_fare -> fare_amount, tips -> tip_amount) to align with shared schema. Adds trip_type='hvfhv'.

models/intermediate/int_all_trips_unioned.sql

Unions Yellow, Green, and HVFHV staging models into a single dataset with a consistent schema and a trip_type discriminator. This is the source for fct_trips - avoids repeating the union in multiple downstream models.

models/dim/dim_taxi_zones.sql

Dimension table for the 265 NYC taxi zones. Built from the taxi_zone_lookup seed. Used for relationships tests on PULocationID and DOLocationID in fct_trips.

models/dim/dim_vendors.sql

Static dimension for taxi vendors (1=CMT, 2=VeriFone). Built from the vendors seed.

models/dim/dim_rate_codes.sql

Static dimension mapping RatecodeID integers to human-readable labels. Built from the rate_codes seed.

models/dim/dim_payment_types.sql

Static dimension mapping payment_type integers to labels. Built from the payment_types seed.

models/fct/fct_trips.sql

Core incremental fact table. Pulls from int_all_trips_unioned, joins to dims, generates trip_id surrogate key via dbt_utils.generate_surrogate_key, derives trip_duration_minutes and avg_speed_mph using safe_divide macro. Incremental strategy: unique_key='trip_id'.

models/marts/mart_daily_summary.sql

Aggregates `fct_trips` by `pickup_date`. Columns: `trip_count`, `avg_fare_amount`, `avg_trip_distance`, `avg_trip_duration_minutes`, `total_revenue`. Primary BI dashboard table for daily trends.

models/marts/mart_zone_performance.sql

Aggregates `fct_trips` by `pu_location_id` joined to `dim_taxi_zones`. Shows zone, borough, `trip_count`, `total_revenue`, `avg_fare`, `avg_distance`. Identifies top and bottom performing pickup zones.

models/marts/mart_revenue_breakdown.sql

Breaks down revenue components by `trip_type` and `pickup_date`: `fare_amount_total`, `tip_amount_total`, `tolls_amount_total`, `surcharge_total`, `airport_fee_total`, `grand_total`. Used for revenue attribution analysis.

macros/safe_divide.sql

Custom macro wrapping Snowflake `NULLIF` to prevent division-by-zero: `{{ numerator }} / NULLIF({{ denominator }}, 0)`. Used in `fct_trips` for `avg_speed_mph` calculation.

8.4 Snowflake & Root Files

snowflake/setup.sql

One-time DDL script run manually in Snowflake before the pipeline starts. Contains: CREATE ROLE TRANSFORM, CREATE WAREHOUSE COMPUTE_WH (AUTO_SUSPEND=300), CREATE DATABASE TLC with RAW/STAGING/MARTS/LOOKUP schemas, GRANT statements, CREATE STAGE pointing to S3, CREATE FILE FORMAT for Parquet, and CREATE TABLE DDL for all 4 raw tables.

.env.example

Template showing all required environment variables with no real values: SNOWFLAKE_ACCOUNT, SNOWFLAKE_USER, SNOWFLAKE_PASSWORD, SNOWFLAKE_WAREHOUSE, SNOWFLAKE_DATABASE, AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, S3_BUCKET, SLACK_WEBHOOK_URL. Committed to Git. The actual .env is in .gitignore.

README.md

Top-level project documentation covering: architecture diagram, prerequisites, step-by-step setup (Snowflake, Airflow Docker, dbt profile, env vars), how to run the full pipeline manually, how to run just dbt, and how to access dbt docs lineage.

9. Key Concepts Covered

Concept	Implementation
Medallion architecture	PySpark (Bronze) -> RAW (Silver) -> MARTS (Gold)
Incremental loading	fct_trips incremental on tpep_pickup_datetime + PySpark watermarks
Surrogate keys	dbt_utils.generate_surrogate_key on fct_trips
Data contracts	dbt generic + dbt_expectations tests on every layer
Idempotent ingestion	COPY INTO with FORCE=FALSE + PySpark dedup
Null-safe division	Custom safe_divide macro wrapping NULLIF
Cost control	Auto-suspend Snowflake warehouse + partitioned Parquet
Secrets management	Airflow Connections + .env.example (never hardcoded)
Observability	Row count logging (Spark) + dbt test results + Slack alerts
Lineage	dbt docs with exposure declarations for BI layer
SCD Type 2	dbt snapshots for historical dimension tracking (optional extension)
Least privilege	Dedicated TRANSFORM role with scoped Snowflake grants